script.js

O “cérebro” do navegador: desenha o mapa, anima as correntes, calcula as 3 rotas e chama o Gemini.

JavaScript · 4.287 linhas · roda no navegador (sem precisar do servidor Python)

Visão geral

O **script.js** é o maior arquivo do projeto. Ele lê os dados de correntes (**ocean-data.js**), projeta o globo num canvas, anima milhares de partículas seguindo as correntes e permite escolher dois portos para comparar três rotas: **comercial** (menor distância), **A*** (menor tempo usando as correntes) e **Copiloto IA** (menor combustível, com velocidade do motor ajustada por trecho). Também simula a deriva de uma mancha de óleo, mostra zonas de cardume e pede o parecer escrito ao Gemini.

Este PDF mostra os blocos mais importantes do código, na ordem em que o programa os usa, e explica o que cada um faz. Cada bloco é copiado do arquivo real, com a faixa de linhas indicada.

Mapa das seções do arquivo

Linhas	Seção	Para que serve
7–80	DOM, configuração e paleta	Pega os canvas do HTML, define densidade de partículas e cores do mapa (claro/escuro).
81–156	Estado	Variáveis globais: zoom, rota atual, navio (velocidade), camadas ligadas.
157–216	Dados oceânicos	Decodifica o ocean-data.js (base64 → números) e interpola a grade.
217–358	Campo sintético	Correntes desenhadas à mão (Golfo, Kuroshio…) usadas se não houver dados reais.
359–504	Projeção e zoom	Converte lat/lon em pixels, controla zoom, arrasto e enquadramento.
505–798	Texturas de satélite	Gera, por código, o visual de oceano e continentes (relèvo, vegetação).
799–1120	Correntes e partículas	Monta o campo de velocidade e anima as partículas a cada quadro.
1121–1425	Camadas do mapa	Desenha base, mancha de óleo, canais bloqueados e mapa de calor dos cardumes.
1426–1766	Tooltips, fronteiras e rotas	Hover das camadas, bordas de países, gelo polar, linhas de rotas e nomes de oceanos.
1767–1996	Portos	Marcadores, cartão com foto (Wikipédia) e menus de origem/destino.
2071–2382	Grade de navegação e A*	Máscara navegável, canais, custo de aresta e algoritmo A*.
2383–2636	As 3 abordagens	Otimizador de potência do Copiloto e comparação das rotas.
2637–2879	Backend e Gemini	Chama /api/route; sem servidor, chama o Gemini direto do navegador.
2880–3166	Emergência e desenho das rotas	Porto seguro mais próximo e linhas coloridas por potência.
3167–3327	Análise da rota	Preenche a tabela comparativa (modal) com os números das 3 rotas.
3328–3665	Simulações IA	Deriva de vazamento de óleo calculada no navegador.
3666–3795	Cardumes e legenda	Liga/desliga a camada de cardumes (servidor ou arquivo pronto).
3796–4198	Botões, tema e controles	Atualizar correntes, tema claro/escuro, HUD e eventos de mouse/toque.
4199–4286	Inicialização	init(): carrega tudo na ordem certa e inicia a animação.

Blocos de código explicados

Bloco 1 — Canvas e configuração das partículas

script.js · linhas 7–27

```
/* ---------- DOM ---------- */
const baseCanvas = document.getElementById('base');
const flowCanvas = document.getElementById('flow');
const routeCanvas = document.getElementById('route');
const bctx = baseCanvas.getContext('2d');
const fctx = flowCanvas.getContext('2d');
const rctx = routeCanvas.getContext('2d');
const warnEl = document.getElementById('warn');
const readoutEl = document.getElementById('readout');
const portTooltipEl = document.getElementById('port-tooltip');
let routeInfoEl = null;

/* ---------- configuração ---------- */
const CFG = { fade: 0.042, speed: 0.16, stepCap: 0.9, density: 360, minP: 1400, maxP: 8800 };

/* ---------- níveis de densidade de partículas (para celulares mais fracos) ---------- */
const DENSITY_LEVELS = {
  alta:  { density: 360,  minP: 1400, maxP: 8800 },
  media: { density: 650,  minP: 700,  maxP: 4200 },
  baixa: { density: 1200, minP: 300,  maxP: 1600 },
};
```

O que este bloco faz

Pega do HTML as três camadas de desenho: **base** (oceano, terra, camadas), **flow** (partículas das correntes) e **route** (linhas das rotas). Separar em canvas diferentes evita redesenhar o mapa inteiro a cada quadro da animação.

CFG controla a animação: *fade* (rápido o rastro some), *speed* (velocidade), *density* (pixels por partícula) e os limites mínimo/máximo de partículas.

Bloco 2 — Decodificação dos dados de correntes

script.js · linhas 164–184

```
function b64ToI16(s) {
  const bin = atob(s);
  const bytes = new Uint8Array(bin.length);
  for (let i = 0; i < bin.length; i++) bytes[i] = bin.charCodeAt(i);
  return new Int16Array(bytes.buffer);
}

function decodeOceanData() {
  const src = window.OCEAN_DATA;
  if (!src) return null;
  const cu = b64ToI16(src.cur.u), cv = b64ToI16(src.cur.v);
  const U = new Float32Array(cu.length), V = new Float32Array(cv.length);
  for (let i = 0; i < cu.length; i++) { U[i] = cu[i] / 1000; V[i] = cv[i] / 1000; }
  let T = null;
  if (src.sst && src.sst.t) {
    const st = b64ToI16(src.sst.t);
    T = new Float32Array(st.length);
    for (let i = 0; i < st.length; i++) T[i] = st[i] / 100;
  }
  return { U, V, T, c: src.cur, s: src.sst || null, source: src.source, regional: false };
}
```

O que este bloco faz

O **ocean-data.js** guarda as correntes como texto base64 de números inteiros (para o arquivo ficar pequeno). Estas funções convertem o texto de volta para números:

- **b64ToI16**: base64 → vetor de inteiros de 16 bits.
- **decodeOceanData**: divide por 1000 (corrente u/v em m/s) e por 100 (temperatura em °C), devolvendo as grades U, V e T.

Bloco 3 — Interpolação bilinear da grade

script.js · linhas 203–215

```
function sampleGrid(arr, g, lon, lat) {
  let x = (lon - g.lon0) / g.dlon;
  x = ((x % g.nlon) + g.nlon) % g.nlon;
  let y = (lat - g.lat0) / g.dlat;
  if (y < 0) y = 0; else if (y > g.nlat - 1) y = g.nlat - 1;
  const xf = Math.floor(x), yf = Math.floor(y);
  const tx = x - xf, ty = y - yf;
  const x0 = xf % g.nlon, x1 = (xf + 1) % g.nlon;
  const y0 = yf, y1 = y0 + 1 < g.nlat ? y0 + 1 : y0;
  const w = g.nlon;
  return (arr[y0 * w + x0] * (1 - tx) + arr[y0 * w + x1] * tx) * (1 - ty) +
         (arr[y1 * w + x0] * (1 - tx) + arr[y1 * w + x1] * tx) * ty;
}
```

O que este bloco faz

Os dados estão numa grade de 0,5°, mas as partículas e o navio ficam em qualquer ponto. Esta função descobre as 4 células vizinhas do ponto e faz a média ponderada pela distância, o que dá uma corrente contínua e suave.

A longitude é tratada de forma circular (o mundo dá a volta), e a latitude é limitada aos polos.

Bloco 4 — Projeção do mapa (lat/lon ↔ pixel)

script.js · linhas 359–374

```
/* ---------- projeção (equiretangular linear) + vista (zoom/pan) ---------- */
function project(lon, lat) { return [originX + (lon + 180) * scaleK, originY + (90 - lat) * scaleK]; }
function invert(x, y) { return [(x - originX) / scaleK - 180, 90 - (y - originY) / scaleK]; }

function clampView() {
  const worldW = 360 * scaleK, worldH = 180 * scaleK;
  originX = worldW <= W ? (W - worldW) / 2 : Math.min(0, Math.max(W - worldW, originX));
  originY = worldH <= H ? (H - worldH) / 2 : Math.min(0, Math.max(H - worldH, originY));
}

function updateProj() {
  proj = d3.geoEquirectangular()
    .scale(scaleK * 180 / Math.PI)
    .translate([originX + 180 * scaleK, originY + 90 * scaleK]);
  geoPath = d3.geoPath(proj, bctx);
}
```

O que este bloco faz

Usa a projeção **equiretangular**: longitude e latitude viram x e y de forma linear. **project** converte coordenadas em pixels e **invert** faz o caminho de volta (usado quando você clica no mapa).

clampView impede arrastar o mapa para fora do mundo e **updateProj** mantém a projeção do D3 sincronizada com zoom e posição.

Bloco 5 — Campo sintético de correntes (plano B)

script.js · linhas 813–868

```
function buildField() {
  for (let j = 0; j < FH; j++) {
    const lat = 90 - (j + 0.5) * (180 / FH);
    for (let i = 0; i < FW; i++) {
      const baseLon = i - 180 + 0.5;
      let u = 0, v = 0;

      for (let ci = 0; ci < CURRENTS.length; ci++) {
        const c = CURRENTS[ci];
        let lon = baseLon;
        const dm = c.meanLon - lon;
        if (dm > 180) lon += 360;
        else if (dm < -180) lon -= 360;

        let best = Infinity, tu = 0, tv = 0;
        for (let k = 0; k < c.pts.length - 1; k++) {
          const ax = c.pts[k][0], ay = c.pts[k][1];
          const dx = c.pts[k + 1][0] - ax, dy = c.pts[k + 1][1] - ay;
          const L2 = dx * dx + dy * dy || 1e-9;
          let t = ((lon - ax) * dx + (lat - ay) * dy) / L2;
          t = t < 0 ? 0 : t > 1 ? 1 : t;
          const ex = lon - (ax + t * dx);
          const ey = lat - (ay + t * dy);
          const d2 = ex * ex + ey * ey;
          if (d2 < best) {
            best = d2;
            const inv = 1 / Math.sqrt(L2);
            tu = dx * inv; tv = dy * inv;
          }
        }
        const w = Math.exp(-best / (c.width * c.width));
        if (w > 0.01) { u += tu * c.speed * w; v += tv * c.speed * w; }
      }

      for (let gi = 0; gi < BG_GYRES.length; gi++) {
        const G = BG_GYRES[gi];
        let dlon = baseLon - G.lon;
        if (dlon > 180) dlon -= 360;
        else if (dlon < -180) dlon += 360;
        const nx = dlon / G.rx, ny = (lat - G.lat) / G.ry;
        const env = Math.exp(-(nx * nx + ny * ny));
        if (env > 0.02) {
          u += -G.dir * G.s * ny * env;
          v += G.dir * G.s * nx * env;
        }
      }

      if (isLandLL(baseLon, lat)) { u *= 0.12; v *= 0.12; }

      const o = j * FW + i;
      FIELD[o * 2] = u;
      FIELD[o * 2 + 1] = v;
      SPD[o] = Math.hypot(u, v);
    }
  }
}
```

O que este bloco faz

Se não houver dados reais, o mapa monta um campo de correntes artificial. Para cada célula do globo:

- soma a influência das grandes correntes desenhadas à mão (cada uma é uma poligonal com largura e velocidade; o peso cai como uma gaussiana com a distância);
- soma os giros oceânicos de fundo (rotação horária/anti-horária);
- reduz a velocidade em 88% se a célula é terra.

O resultado vai para os vetores **FIELD** (u, v) e **SPD** (velocidade).

Bloco 6 — Nascimento das partículas

script.js · linhas 986–1020

```
function resetParticle(p) {
  if (CURRENTS.length && !(OD && OD.regional) && Math.random() < 0.4) {
    const c = CURRENTS[(Math.random() * CURRENTS.length) | 0];
    const k = (Math.random() * (c.pts.length - 1)) | 0;
    const tt = Math.random();
    let lon = c.pts[k][0] + (c.pts[k + 1][0] - c.pts[k][0]) * tt + (Math.random() - 0.5) * 2 * c.width;
    let lat = c.pts[k][1] + (c.pts[k + 1][1] - c.pts[k][1]) * tt + (Math.random() - 0.5) * 2 * c.width;
    lon = ((lon + 180) % 360 + 360) % 360 - 180;
    p.lon = lon;
    p.lat = lat > 84 ? 84 : lat < -84 ? -84 : lat;
  } else {
    const b = viewBounds();
    for (let k = 0; k < 25; k++) {
      let lon = b.lon0 + Math.random() * (b.lon1 - b.lon0);
      const lat = b.lat0 + Math.random() * (b.lat1 - b.lat0);
      lon = ((lon + 180) % 360 + 360) % 360 - 180;
      if (!isLandLL(lon, lat) || k === 24) { p.lon = lon; p.lat = lat; break; }
    }
  }
  p.age = 0;
  p.maxAge = 14 + Math.random() * 70;
}

function seedParticles() {
  let n = Math.round(W * H / CFG.density);
  n = Math.max(CFG.minP, Math.min(CFG.maxP, n));
  particles = new Array(n);
  for (let i = 0; i < n; i++) {
    const p = {};
    resetParticle(p);
    p.age = Math.random() * p.maxAge;
    particles[i] = p;
  }
  fctx.clearRect(0, 0, W, H);
}
```

O que este bloco faz

Cada partícula é um ponto que “nada” seguindo a corrente e some depois de um tempo. **resetParticle** faz a partícula renascer: 40% das vezes perto das grandes correntes (para o fluxo ficar visível), e nas outras em um ponto aleatório de água dentro da área visível. Define também a vida máxima (14 a 84 quadros).

seedParticles calcula quantas partículas cabem na tela (limitadas por CFG) e espalha as idades para não nascerem todas juntas.

Bloco 7 — Loop de animação das correntes

script.js · linhas 1030–1075

```
function frame(now) {
  rafId = requestAnimationFrame(frame);
  let dt = (now - lastT) / 16.67;
  lastT = now;
  if (dt > 2.5) dt = 2.5; else if (dt < 0.4) dt = 0.4;
  phase += dt;

  fctx.globalCompositeOperation = 'destination-out';
  fctx.fillStyle = 'rgba(0,0,0,' + CFG.fade + ')';
  fctx.fillRect(0, 0, W, H);
  fctx.globalCompositeOperation = 'source-over';

  if (!showCurr) return;

  for (let s = 0; s < 9; s++) SEGS[s].length = 0;

  const cap = CFG.stepCap * dt;
  const gamma = OD ? 1.0 : 1.12;
  const calm = OD ? 0.03 : 0.05;
  const sFast = OD ? 0.85 : 1.7, sMid = OD ? 0.28 : 0.7;

  for (let i = 0; i < particles.length; i++) {
    const p = particles[i];

    const f = sampleField(p.lon, p.lat);
    const inReg = !ODB || (p.lon >= ODB.lon0 && p.lon <= ODB.lon1 &&
                           p.lat >= ODB.lat0 && p.lat <= ODB.lat1);
    const tb = inReg ? streamTurb(p.lon, p.lat, phase) : [0, 0];
    const ls = Math.hypot(f[0], f[1]);
    const g = OD ? (0.05 + 0.12 * Math.max(0, 1 - ls / 0.4))
                 : (0.30 + 0.8 * Math.min(ls / 2.2, 1));
    const u = f[0] + tb[0] * g;
    const v = f[1] + tb[1] * g;
    const sp = Math.hypot(u, v);

    p.age += dt * (ls < calm ? 2.2 : 1);

    if (sp < 1e-4 || p.age > p.maxAge) { resetParticle(p); continue; }

    let step = Math.pow(sp, gamma) * CFG.speed * dt;
    if (step > cap) step = cap;

    const nlon = p.lon + (u / sp) * step;
    const nlat = p.lat + (v / sp) * step;

    if (nlon > 180 || nlon < -180 || nlat > 87 || nlat < -87 || isLandLL(nlon, nlat)) {
```

O que este bloco faz

Roda uma vez por quadro (*requestAnimationFrame*). A cada quadro:

- apaga um pouco o desenho anterior (*destination-out*) para criar o rastro das partículas;
- para cada partícula, amostra a corrente no ponto, soma uma turbulência leve e calcula o passo seguinte, limitado por *stepCap*;
- se a partícula sai do mapa, bate na terra ou envelhece demais, ela renasce;
- guarda os segmentos em 9 grupos (3 temperaturas × 3 velocidades) para desenhar tudo de uma vez, o que é bem mais rápido.

Bloco 8 — Grade de navegação (onde o navio pode ir)

script.js · linhas 2080–2095

```
function buildNav() {
  if (OD && OD.regional && ODB) {
    NRES = OD.c.dlon;
    N_LON0 = OD.c.lon0;
    N_LAT0 = ODB.lat1;
    N_WRAP = false;
    NLON = OD.c.nlon;
    NLAT = OD.c.nlat;
  }
  NAV_RAW = new Uint8Array(NLON * NLAT);
  for (let a = 0; a < NLAT; a++) {
    const lat = navLat(a);
    for (let o = 0; o < NLON; o++) {
      NAV_RAW[navIdx(a, o)] = (Math.abs(lat) <= 84 && !isLandLL(navLon(o), lat)) ? 1 : 0;
    }
  }
```

O que este bloco faz

Cria uma grade onde cada célula vale 1 (mar navegável) ou 0 (terra ou latitude acima de 84°). É nessa grade que o A* procura caminho. Ao final aplica o cenário atual (canais abertos ou fechados).

Se o mapa estiver em modo regional, a grade usa a resolução dos dados regionais.

Bloco 9 — Estreitos e canais (Panamá e Suez)

script.js · linhas 2103–2118

```
const NAV_STRAITS = {
  'Estreito de Dover': [[51, 2], [51, 1], [50, 1], [50, 0], [50, -1]],
  'Estreito de Juan de Fuca': [[48, -123], [48, -124], [48, -125], [48, -126]],
  'Estreitos dinamarqueses': [[58, 10], [57, 10], [57, 11], [56, 11], [56, 12], [55, 12], [55, 13], [55, 14], [55, 15], [55,
      ↳ 16]],
  'Bósforo e Dardanelos': [[42, 29], [41, 29], [41, 28], [40, 28], [40, 27], [40, 26], [39, 26], [39, 25]],
  'Mar Egeu': [[40, 23], [40, 24], [39, 24], [38, 24], [38, 25], [37, 25], [36, 25], [35, 25]],
  'Bab-el-Mandeb': [[14, 42], [13, 42], [13, 43], [12, 43], [12, 44], [12, 45]],
  'Estuário do São Lourenço': [[49, -68], [49, -67], [49, -66], [50, -66], [50, -65]],
  'Golfo da Finlândia': [[58, 20], [59, 20], [59, 21], [59, 22], [59, 23], [59, 24], [60, 24], [60, 25], [60, 26], [60, 27],
      ↳ [60, 28]],
  'Golfo de Honduras': [[16, -88], [16, -87], [16, -86], [17, -86], [17, -85], [17, -84]],
  'Estreito de Malaca': [[6, 96], [6, 97], [5, 97], [5, 98], [4, 98], [4, 99], [3, 99], [3, 100], [2, 100], [2, 101], [2, 102],
      ↳ [1, 102], [1, 103], [1, 104], [1, 105]],
};
const NAV_CANALS = {
  panama: [[12, -80], [11, -80], [10, -80], [9, -80], [9, -79], [8, -79], [7, -79], [6, -79]],
  suez: [[31, 32], [30, 32], [29, 32], [29, 33], [28, 33], [27, 33], [27, 34]],
};
```

O que este bloco faz

A grade de 1° “perde” estreitos pequenos (Dóver, Bósforo, Malaca…), por isso a lista **NAV_STRAITS** mantém corredores sempre abertos.

NAV_CANALS guarda as células dos canais do Panamá e de Suez: o usuário pode “bloquear” cada um na tela, e o A* passa a contornar o continente. O Ártico acima de 73°N fica fechado (gelo marinho).

Bloco 10 — Distância e custo de aresta do A* (tempo real)

script.js · linhas 2208–2244

```
function havKm(lo1, la1, lo2, la2) {
  const p1 = la1 * Math.PI / 180, p2 = la2 * Math.PI / 180;
  const dp = (la2 - la1) * Math.PI / 180;
  let dl = ((lo2 - lo1 + 540) % 360 - 180) * Math.PI / 180;
  const s = Math.sin(dp / 2) ** 2 + Math.cos(p1) * Math.cos(p2) * Math.sin(dl / 2) ** 2;
  return 2 * R_EARTH * Math.asin(Math.min(1, Math.sqrt(s)));
}

/* Custo de aresta do A* = tempo real: Tempo = Distância / |vnavio + vcorrente|.
   O navio aproa contra a componente transversal da corrente e avança a
   sqrt(v² − c⊥²) + c? sobre o fundo. kn = velocidade na água (padrão: a de serviço);
   cw = peso da corrente (0 = planejamento tradicional, que ignora o mar). */
function edgeHours(lo1, la1, lo2, la2, kn, cw) {
  if (kn == null) kn = SHIP.kn;
  if (cw == null) cw = 1;
  const latM = (la1 + la2) / 2;
  let dLon = ((lo2 - lo1 + 540) % 360) - 180;
  const dx = dLon * 111.32 * Math.cos(latM * Math.PI / 180);
  const dy = (la2 - la1) * 110.57;
  const len = Math.hypot(dx, dy);
  if (len < 1e-6) return 0;
  const ex = dx / len, ey = dy / len;

  const f = sampleField(lo1 + dLon / 2, latM);
  const S = (OD ? 3.6 : 2.0) * cw;
  const cu = f[0] * S;
  const cv = f[1] * S;
  const cPar = cu * ex + cv * ey;
  const cPerp2 = Math.max(0, cu * cu + cv * cv - cPar * cPar);

  const Vs = kn * KMH_PER_KN;
  const avail = Vs * Vs - cPerp2;
  if (avail <= 1) return Infinity;
  const ground = Math.sqrt(avail) + cPar;
  if (ground < 0.5) return Infinity;
  return len / ground;
}
```

O que este bloco faz

havKm calcula a distância em km entre dois pontos da Terra (fórmula de Haversine).

edgeHours é o custo do A*: o **tempo** para atravessar um trecho. O navio compensa a corrente lateral e avança sobre o fundo a $\sqrt{(v^2 - c_\perp^2)} + c_\parallel$, onde v é a velocidade do navio, $c_\perp$ a corrente transversal e $c_\parallel$ a corrente a favor ou contra. Resultado: tempo = distância / velocidade sobre o fundo.

Se a corrente for forte demais para o navio avançar, retorna *Infinity* (trecho proibido). O parâmetro *cw* liga (1) ou desliga (0) o efeito da corrente — com 0 vira a rota comercial.

Bloco 11 — Fila de prioridade (heap mínimo)

script.js · linhas 2246–2278

```
class MinHeap {
  constructor() { this.k = []; this.p = []; }
  get size() { return this.k.length; }
  push(key, pri) {
    this.k.push(key); this.p.push(pri);
    let i = this.k.length - 1;
    while (i > 0) {
      const par = (i - 1) >> 1;
      if (this.p[par] <= this.p[i]) break;
      this._sw(i, par); i = par;
    }
  }
  pop() {
    const top = this.k[0];
    const lk = this.k.pop(), lp = this.p.pop();
    if (this.k.length) {
      this.k[0] = lk; this.p[0] = lp;
      let i = 0;
      for (;;) {
        let s = i; const l = 2 * i + 1, r = 2 * i + 2;
        if (l < this.k.length && this.p[l] < this.p[s]) s = l;
        if (r < this.k.length && this.p[r] < this.p[s]) s = r;
        if (s === i) break;
        this._sw(i, s); i = s;
      }
    }
    return top;
  }
  _sw(a, b) {
    const tk = this.k[a]; this.k[a] = this.k[b]; this.k[b] = tk;
    const tp = this.p[a]; this.p[a] = this.p[b]; this.p[b] = tp;
  }
}
```

O que este bloco faz

Estrutura que o A* usa para sempre retirar primeiro a célula mais promissora (menor custo estimado). É um heap binário feito à mão: **push** insere e “sobe” o item; **pop** remove o menor e reorganiza. Isso mantém a busca rápida mesmo com centenas de milhares de células.

Bloco 12 — Algoritmo A* na grade

script.js · linhas 2284–2339

```
function aStar(aS, oS, aG, oG, cw, custom) {
  const N = NLON * NLAT;
  const g = new Float64Array(N).fill(Infinity);
  const came = new Int32Array(N).fill(-1);
  const closed = new Uint8Array(N);
  const startI = navIdx(aS, oS), goalI = navIdx(aG, oG);
  const goalLon = navLon(oG), goalLat = navLat(aG);
  const maxGround = (SHIP.kn + 6) * KMH_PER_KN;
  const edgeCost = custom ? custom.cost : (lo1, la1, lo2, la2) => edgeHours(lo1, la1, lo2, la2, SHIP.kn, cw);
  const heur = custom ? custom.heur : (km) => km / maxGround;
  g[startI] = 0;
  const heap = new MinHeap();
  heap.push(startI, heur(havKm(navLon(oS), navLat(aS), goalLon, goalLat)));

  let guard = 0;
  while (heap.size) {
    if (++guard > 800000) break;
    const cur = heap.pop();
    if (cur === goalI) break;
    if (closed[cur]) continue;
    closed[cur] = 1;
    const ca = (cur / NLON) | 0, co = cur % NLON;
    const clon = navLon(co), clat = navLat(ca);
    for (let n = 0; n < 8; n++) {
      const na = ca + NB8[n][0];
      if (na < 0 || na >= NLAT) continue;
      let no = co + NB8[n][1];
      if (N_WRAP) no = ((no % NLON) + NLON) % NLON;
      else if (no < 0 || no >= NLON) continue;
      const ni = na * NLON + no;
      if (!NAV[ni] || closed[ni]) continue;
      if (NB8[n][0] !== 0 && NB8[n][1] !== 0) {
        if (!NAV[ca * NLON + no] || !NAV[na * NLON + co]) continue;
      }
      const dc = edgeCost(clon, clat, navLon(no), navLat(na));
      if (!isFinite(dc)) continue;
      const ng = g[cur] + dc;
      if (ng < g[ni]) {
        g[ni] = ng;
        came[ni] = cur;
        heap.push(ni, ng + heur(havKm(navLon(no), navLat(na), goalLon, goalLat)));
      }
    }
  }
  if (goalI !== startI && came[goalI] === -1) return null;
  const path = [];
  let c = goalI;
  while (c !== -1) {
    path.push([navLon(c % NLON), navLat((c / NLON) | 0)]);
    if (c === startI) break;
    c = came[c];
  }
  path.reverse();
  if (custom) return { path, cost: g[goalI] };
  return { path, hours: g[goalI] };
}
```

O que este bloco faz

O coração do projeto. Procura o caminho de custo mínimo entre duas células:

- **g** guarda o custo acumulado até cada célula; **came** guarda de qual célula chegamos (para reconstruir o caminho);
- a prioridade na fila é **g + heur**: a heurística estima o tempo restante em linha reta à velocidade máxima possível, por isso nunca superestima e o resultado é ótimo;
- cada célula olha seus 8 vizinhos, ignora terra e não permite “cortar” diagonais entre duas células de terra;
- o parâmetro *custom* troca a função de custo (tempo) por outra (combustível) — é assim que o Copiloto IA reaproveita o mesmo A*.

Bloco 13 — Rota do Copiloto IA: custo em combustível

script.js · linhas 2345–2381

```
const AI_ADVERSE_WEIGHT = 1.5;
const AI_CROSS_PENALTY = 0.8;
const AI_MODE_LABEL = '100% Eco / Slow Steaming';

function aiAStar(aS, oS, aG, oG) {
  const vPlan = Math.max(6, SHIP.kn * ECO_ETA_FRAC);
  const vs = vPlan * KMH_PER_KN;
  const rate = fuelTonsPerHour(vPlan);
  const rateService = fuelTonsPerHour(SHIP.kn);
  const weight = AI_ADVERSE_WEIGHT;
  const crossW = AI_CROSS_PENALTY;
  const S = OD ? 3.6 : 2.0;
  const cost = (lo1, la1, lo2, la2) => {
    const latM = (la1 + la2) / 2;
    const dLon = ((lo2 - lo1 + 540) % 360) - 180;
    const dx = dLon * 111.32 * Math.cos(latM * Math.PI / 180);
    const dy = (la2 - la1) * 110.57;
    const len = Math.hypot(dx, dy);
    if (len < 1e-6) return 0;
    const f = sampleField(lo1 + dLon / 2, latM);
    const cu = f[0] * S, cv = f[1] * S;
    const cPar = (cu * dx + cv * dy) / len;
    const cPerp = Math.sqrt(Math.max(0, cu * cu + cv * cv - cPar * cPar));
    const avail = vs * vs - cPerp * cPerp;
    if (avail <= 1) return Infinity;
    const ground = Math.sqrt(avail) + cPar;
    if (ground < 0.5) return Infinity;
    const dt = len / ground;
    const adverse = Math.pow((vs + weight * Math.max(0, -cPar)) / vs, 3);
    const fuel = rate * dt * adverse * (1 + crossW * cPerp / vs);
    return fuel / rateService;
  };
  const maxGround = vs + 6 * KMH_PER_KN;
  const perKm = (rate / rateService) / maxGround; // penalidades >= 1: admissível
  const r = aStar(aS, oS, aG, oG, 1, { cost, heur: (km) => perKm * km });
  return r ? { path: r.path, hours: pathHours(r.path, SHIP.kn) } : null;
}
```

O que este bloco faz

Define o custo de cada trecho para a rota **100% Eco**. Em vez de tempo, o custo é o **combustível** gasto, que cresce com o cubo da velocidade. Para evitar gastar muito, o custo ainda recebe duas penalidades:

- **corrente contrária**: multiplica o consumo por $((v + 1{,}5 \cdot c_{contra})/v)^3$;
- **corrente transversal**: aumenta o consumo em 80% × $c_{\perp}/v$.

Com isso o A* desvia das correntes que atrapalham e aproveita as que ajudam, mesmo que a rota fique um pouco mais longa.

Bloco 14 — Constantes da gestão de potência

script.js · linhas 2386–2400

```
const MIN_SPEED_FRAC = 0.60;  // piso do motor por trecho (60% da velocidade de serviço)
const ECO_ETA_FRAC = 0.75;    // ETA-alvo 100% Eco = trajeto a 75% da velocidade de serviço
const SPEED_STEP_KN = 0.25;

// tempo REAL de um trecho já traçado (a corrente age mesmo que o planejamento a ignore)
function segmentHours(lo1, la1, lo2, la2, kn) {
  const dt = edgeHours(lo1, la1, lo2, la2, kn, 1);
  return isFinite(dt) ? dt : havKm(lo1, la1, lo2, la2) / (0.25 * kn * KMH_PER_KN);
}

function pathHours(p, kn) {
  let t = 0;
  for (let i = 1; i < p.length; i++) t += segmentHours(p[i - 1][0], p[i - 1][1], p[i][0], p[i][1], kn);
  return t;
}
```

O que este bloco faz

MIN_SPEED_FRAC = 0,60: o motor nunca vai abaixo de 60% da velocidade de serviço em um trecho.

ECO_ETA_FRAC = 0,75: o prazo-alvo do modo Eco equivale a fazer a viagem a 75% da velocidade.

segmentHours / **pathHours** calculam o tempo *real* de um trajeto (a corrente age mesmo que a rota tenha sido planejada ignorando-a).

Bloco 15 — Otimizador de potência do motor

script.js · linhas 2405–2467

```
function optimizePower(p, kn, etaBudget) {
  const vmin = Math.max(6, kn * MIN_SPEED_FRAC);
  const speeds = [];
  for (let v = kn; v >= vmin - 1e-9; v -= SPEED_STEP_KN) speeds.push(Math.round(v * 100) / 100);

  const segs = [], segKm = [];
  for (let i = 1; i < p.length; i++) {
    const [lo1, la1] = p[i - 1], [lo2, la2] = p[i];
    const opts = [];
    for (const v of speeds) {
      const t = edgeHours(lo1, la1, lo2, la2, v, 1);
      if (isFinite(t)) opts.push([v, t, fuelTonsPerHour(v) * t]);
    }
    if (!opts.length) {
      const t = segmentHours(lo1, la1, lo2, la2, kn);
      opts.push([kn, t, fuelTonsPerHour(kn) * t]);
    }
    segs.push(opts);
    segKm.push(havKm(lo1, la1, lo2, la2));
  }

  const solve = (lam) => {
    let T = 0;
    const pick = segs.map((o) => {
      let best = o[0], bc = o[0][2] + lam * o[0][1];
      for (let k = 1; k < o.length; k++) {
        const c = o[k][2] + lam * o[k][1];
        if (c < bc) { bc = c; best = o[k]; }
      }
      T += best[1];
      return best;
    });
    return [pick, T];
  };

  let [pick, T] = solve(0);
  if (T > etaBudget) {
    let lo = 0, hi = 1;
    while (solve(hi)[1] > etaBudget && hi < 1e6) hi *= 2;
    for (let it = 0; it < 45; it++) {
      const mid = (lo + hi) / 2;
      if (solve(mid)[1] > etaBudget) lo = mid; else hi = mid;
    }
    [pick, T] = solve(hi);
  }

  const fuel = pick.reduce((s, x) => s + x[2], 0);
  const km = segKm.reduce((s, x) => s + x, 0);
  const v = pick.map((x) => x[0]);
  let ecoKm = 0;
  v.forEach((sv, i) => { if (sv < kn - 0.01) ecoKm += segKm[i]; });
  return {
    km, hours: T, fuel_tons: fuel, co2_tons: fuel * CO2_PER_TON_FUEL,
    speeds: v,
    avg_kn: T > 0 ? pick.reduce((s, x) => s + x[0] * x[1], 0) / T : kn,
    min_kn: v.length ? Math.min(...v) : kn,
    max_kn: v.length ? Math.max(...v) : kn,
    load_pct: T > 0 ? 100 * fuel / (fuelTonsPerHour(kn) * T) : 100,
    eco_share: km > 0 ? ecoKm / km : 0,
    eta_budget_h: etaBudget,
    service_kn: kn,
  };
}
```

O que este bloco faz

Decide a velocidade de cada trecho da rota para **gastar o mínimo de combustível** sem estourar o prazo (ETA). Como o consumo cresce com v³, vale reduzir a potência onde a corrente ajuda e manter onde ela atrapalha.

- Para cada trecho, testa várias velocidades (de 18 até 10,8 nós, em passos de 0,25) e anota tempo e combustível de cada uma.
- **solve(λ)** escolhe, em cada trecho, a opção que minimiza combustível + λ·tempo. O λ é o “preço de uma hora”.
- Uma **bisseção** (45 passos) ajusta o λ até o tempo total caber no ETA.
- No fim devolve distância, horas, combustível, CO_2, velocidade mínima/média/máxima e a carga média do motor.

Bloco 16 — Texto de análise local (prévia)

script.js · linhas 2469–2489

```
function localAnalysis(comm, astar, ai) {
  const dFuel = Math.max(0, comm.fuel_tons - ai.fuel_tons);
  const pct = comm.fuel_tons > 0 ? 100 * dFuel / comm.fuel_tons : 0;
  const eta = ai.hours - comm.hours;
  const etaTxt = eta < -0.05 ? 'chegando ' + nf1(-eta) + ' h antes da rota comercial'
    : eta > 0.05 ? 'com ETA ' + nf1(eta) + ' h após a rota comercial'
    : 'mantendo o mesmo ETA da rota comercial';
  const dKm = ai.km - astar.km;
  const rotaTxt = Math.abs(dKm) > 1
    ? 'traça um desvio próprio ' + nf1(Math.abs(dKm) * 0.539957) + ' NM ' + (dKm > 0 ? 'mais longo' : 'mais curto') +
      ' que o A* para fugir de correntes contrárias e transversais'
    : 'segue praticamente o mesmo corredor do A* (correntes fracas nesta travessia)';
  return 'Análise técnica (prévia local — aguardando o parecer do Copiloto IA/Gemini, a tabela atualiza sozinha): o A* ganha ' +
    nf1(comm.hours - astar.hours) + ' h sobre a rota comercial explorando as correntes; o Copiloto ' +
    rotaTxt + ', variando a velocidade entre ' + nf1(ai.min_kn) + ' e ' +
    nf1(ai.max_kn) + ' nós (média ' + nf1(ai.avg_kn) + ' nós, carga média ' + Math.round(ai.load_pct) +
    '% da potência de serviço) e reduzindo o motor em ' + Math.round(100 * ai.eco_share) +
    '% do trajeto. Resultado: −' + nf1(dFuel) + ' t de VLSFO (−' + Math.round(pct) + '%), −' +
    nf1(dFuel * CO2_PER_TON_FUEL) + ' t de CO₂ e US$ ' + Math.round(dFuel * FUEL_PRICE_USD).toLocaleString('pt-BR') +
    ' a menos, ' + etaTxt + '.';
}
```

O que este bloco faz

Monta um parecer em texto com os números já calculados (economia de combustível, CO_2, dinheiro e impacto no ETA). É mostrado na hora, enquanto o parecer do Gemini ainda está sendo gerado; quando ele chega, substitui este texto.

Bloco 17 — Comparação das 3 abordagens

script.js · linhas 2492–2506

```
function buildRouteComparison(astarRes, commRes, aiRes) {
  const kn = SHIP.kn, rate = fuelTonsPerHour(kn);
  const commHours = pathHours(commRes.path, kn);
  let a = astarRes;
  if (commHours < a.hours) a = { path: commRes.path, hours: commHours }; // A* nunca pior que a comercial
  const aiPath = aiRes || a; // sem traçado próprio, o Copiloto usa o caminho do A*
  const comm = { path: commRes.path, km: pathKm(commRes.path), hours: commHours, fuel_tons: rate * commHours, service_kn: kn };
  const astar = { path: a.path, km: pathKm(a.path), hours: a.hours, fuel_tons: rate * a.hours, service_kn: kn };
  // ETA-alvo da gestão de potência (100% Eco): até o piso de 75% da velocidade no trajeto inteiro
  const eta = aiPath.hours / ECO_ETA_FRAC;
  const ai = Object.assign({ path: aiPath.path },
    optimizePower(aiPath.path, kn, eta));
  ai.rationale = localAnalysis(comm, astar, ai);
  return { baseline: comm, astar, llm: ai };
}
```

O que este bloco faz

Junta as três rotas num só objeto (*baseline*, *astar*, *llm*):

- calcula o tempo e o combustível da rota comercial (velocidade constante);
- garante que o A* nunca seja pior que a comercial;
- para o Copiloto, chama **optimizePower** com o ETA-alvo e gera o texto de análise.

Bloco 18 — Gemini direto do navegador

script.js · linhas 2654–2697

```
const GEMINI_KEY_STORE = 'copiloto_gemini_key';
const BROWSER_GEMINI_MODELS = ['gemini-flash-latest', 'gemini-3.8-flash', 'gemini-flash-lite-latest'];

function getBrowserGeminiKey() {
  let k = '';
  try { k = (localStorage.getItem(GEMINI_KEY_STORE) || '').trim(); } catch (e) { /* sem storage */ }
  // sem chave própria do visitante: usa a chave do projeto (config.js)
  return k || String(window.COPILOTO_GEMINI_KEY || '').trim();
}
function setBrowserGeminiKey(k) {
  try { if (k) localStorage.setItem(GEMINI_KEY_STORE, k.trim()); else localStorage.removeItem(GEMINI_KEY_STORE); } catch (e) {
      ↳ /* sem storage */ }
}

async function browserGeminiText(prompt) {
  const apiKey = getBrowserGeminiKey();
  if (!apiKey) return null;
  for (const model of BROWSER_GEMINI_MODELS) {
    const ctl = new AbortController();
    const timer = setTimeout(() => ctl.abort(), 20000);
    try {
      const res = await fetch('https://generativelanguage.googleapis.com/v1beta/models/' + model +
        ':generateContent?key=' + encodeURIComponent(apiKey), {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ contents: [{ parts: [{ text: prompt }] }] }),
        signal: ctl.signal,
      });
      if (!res.ok) {
        console.warn('[Gemini navegador]', model, res.status, (await res.text()).slice(0, 300));
        if (res.status === 400 || res.status === 401 || res.status === 403) return null; // chave inválida: não insiste
        continue;
      }
      const j = await res.json();
      const txt = j && j.candidates && j.candidates[0] && j.candidates[0].content &&
        j.candidates[0].content.parts && j.candidates[0].content.parts[0].text;
      if (txt && txt.trim()) return txt.trim();
    } catch (e) {
      console.warn('[Gemini navegador]', model, e && e.message);
    } finally {
      clearTimeout(timer);
    }
  }
  return null;
}
```

O que este bloco faz

Faz o site funcionar **sem servidor Python** (GitHub Pages ou index.html aberto direto).

- **getBrowserGeminiKey**: usa a chave salva pelo visitante no navegador; se não houver, usa a do **config.js**.
- **browserGeminiText**: tenta cada modelo da lista com limite de 20 s. Se a chave for recusada (erro 400/401/403), para na hora; se for cota ou erro temporário, tenta o próximo modelo.

Bloco 19 — Monta o pedido ao Gemini

script.js · linhas 2699–2717

```
async function browserGeminiRationale(key) {
  const v = routeVariants;
  if (!v || v.key !== key || !v.baseline || !v.llm) return false;
  const c = v.baseline, a = v.astar, ai = v.llm, NM = 0.539957;
  const f1 = (x) => (isFinite(x) ? (+x).toFixed(1) : '?');
  const origem = routeAPort ? routeAPort.name : 'origem', destino = routeBPort ? routeBPort.name : 'destino';
  const prompt =
    'Atue como um Especialista em Logística Marítima. Análise da rota ' + origem + ' -> ' + destino + ':\n' +
    '- Rota Comercial (menor distância, velocidade constante): ' + f1(c.km * NM) + ' NM, ' + f1(c.hours) + ' h, ' +
        ↳ f1(c.fuel_tons) + ' t VLSFO.\n' +
    '- A* (menor tempo com correntes): ' + f1(a.km * NM) + ' NM, ' + f1(a.hours) + ' h, ' + f1(a.fuel_tons) + ' t VLSFO.\n' +
    '- Copiloto IA (100% Eco, potência ajustada por trecho entre ' + f1(ai.min_kn) + ' e ' + f1(ai.max_kn) + ' nós): ' +
    f1(ai.km * NM) + ' NM, ' + f1(ai.hours) + ' h, ' + f1(ai.fuel_tons) + ' t VLSFO.\n' +
    'Combustível a US$ 600/t e 3,114 t de CO2 por tonelada. Forneça um parecer técnico sucinto em português ' +
    '(máximo 3 frases) destacando economia de combustível, emissões e impacto no ETA. Use só esses números.';
  const txt = await browserGeminiText(prompt);
  if (!txt || routeVariants !== v) return false;
  v.llm.rationale = txt;
  return true;
}
```

O que este bloco faz

Escreve o prompt com os números reais das três rotas (distância, horas, combustível) e pede um parecer técnico de até 3 frases. Se o Gemini responder, o texto é gravado em *routeVariants.llm.rationale*, que é o que a tabela exibe.

Bloco 20 — Servidor ou navegador: quem responde

script.js · linhas 2719–2759

```
async function fetchRouteVariants(a, b) {
  const key = routeVariantsKey(a, b);
  if (!key) return;
  const useBrowserAi = !!getBrowserGeminiKey();
  if (location.protocol === 'file:' && !useBrowserAi) return; // sem servidor e sem chave: fica o texto local
  aiAnalysisPendingKey = key;
  refreshOpenAnalytics(); // modal aberto: troca o texto antigo pela animação de carregamento
  let serverAnswered = false;
  try {
    if (location.protocol === 'file:') throw new Error('sem servidor');
    const res = await fetch('/api/route', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({
        a, b, speed_kn: SHIP.kn, scenario: navScenario,
        origin_name: routeAPort ? routeAPort.name : null,
        dest_name: routeBPort ? routeBPort.name : null,
      }),
    });
    if (!res.ok) throw new Error('servidor ausente'); // ex.: GitHub Pages (site estático)
    const data = await res.json();
    serverAnswered = true;
    if (routeVariantsKey(routeA, routeB) !== key) return; // rota mudou enquanto a resposta chegava
    if (!data.llm || data.llm.service_kn !== SHIP.kn) return; // velocidade mudou no meio do caminho
    routeVariants = prepareVariants(Object.assign({ key, source: 'server' }, data));
    if (!routeIsEmergency) writeRouteSummary(routeVariants);
    drawRoute();
  } catch (e) {
    // sem servidor Python: mapa continua com a rota calculada no cliente; se houver chave, o parecer vem do Gemini direto
    if (!serverAnswered && useBrowserAi) {
      try { await browserGeminiRationale(key); } catch (e2) { /* fica o texto local */ }
    }
  }
  finally {
    // sucesso ou falha: tira a animação e mostra o parecer final (do Gemini ou o texto local)
    if (aiAnalysisPendingKey === key) {
      aiAnalysisPendingKey = null;
      refreshOpenAnalytics();
    }
  }
}
```

O que este bloco faz

Escolhe de onde vem a resposta mais completa:

- Com o servidor Python: chama **/api/route** e usa os números e o texto dele.
- Sem servidor (erro ou arquivo local): mantém os números calculados no navegador e, havendo chave, pede o parecer ao Gemini direto.
- O bloco *finally* sempre remove a animação de carregamento, mesmo que tudo falhe, e atualiza a janela de análise se ela estiver aberta.

Bloco 21 — Traçar a rota entre dois portos

script.js · linhas 2761–2788

```
function computeRoute() {
  if (!routeA || !routeB || !NAV) return;
  routeIsEmergency = false;
  routeVariants = null;
  routeInfoEl.textContent = 'calculando rota…';
  const s = navSnap(routeA[0], routeA[1]);
  const gg = navSnap(routeB[0], routeB[1]);
  if (!s || !gg) { routePath = null; routeInfoEl.textContent = 'ponto fora do oceano navegável.'; drawRoute(); return; }

  const res = aStar(s[0], s[1], gg[0], gg[1]);
  if (!res || res.path.length < 2) {
    routePath = null;
    routeInfoEl.textContent = 'não achei rota — os pontos estão separados por terra?';
    drawRoute();
    return;
  }

  routePath = withEndpoints(res.path);
  const cmp = setClientComparison(s, gg, res);
  if (cmp) {
    writeRouteSummary(cmp);
  } else {
    routeInfoEl.textContent = 'A* (correntes)\n  ' + fmtKm(pathKm(routePath)) + '  ·  ' + fmtDur(res.hours);
  }
  fitViewToPath(routePath, 80);
  drawRoute();
  fetchRouteVariants(routeA, routeB);
}
```

O que este bloco faz

Fluxo principal ao clicar em “Traçar Rota”: encaixa origem e destino na água navegável mais próxima, roda o A*, calcula as três comparações, enquadra o mapa na rota, desenha e, por fim, chama **fetchRouteVariants** para buscar o parecer da IA.

Bloco 22 — Preenche a lista de portos

script.js · linhas 1955–1978

```
function populatePortSelectors() {
  const groups = ['Grande Hub', 'Porto Regional', 'Ancoradouro/Terminal'];
  const sorted = groups.map((g) => ({
    label: g,
    ports: PORTS.filter((p) => p.size === g).sort((a, b) => a.name.localeCompare(b.name)),
  }));

  for (const selId of ['r-origin', 'r-dest']) {
    const sel = document.getElementById(selId);
    if (!sel) continue;
    for (const g of sorted) {
      if (!g.ports.length) continue;
      const og = document.createElement('optgroup');
      og.label = g.label;
      for (const p of g.ports) {
        const opt = document.createElement('option');
        opt.value = p.id;
        opt.textContent = p.name + ' — ' + p.country;
        og.appendChild(opt);
      }
      sel.appendChild(og);
    }
  }
}
```

O que este bloco faz

Monta os menus de Origem e Destino a partir do **global-ports-data.js**, agrupando por tamanho (Grande Hub, Porto Regional, Ancoradouro/Terminal) e em ordem alfabética.

Bloco 23 — Rota de emergência

script.js · linhas 2937–2979

```
function computeEmergencyRoute(lon, lat) {
  const alertEl = document.getElementById('emg-alert');
  routeVariants = null;
  const s = navSnap(lon, lat);
  if (!s || !NAV) {
    if (alertEl) alertEl.textContent = 'Ponto fora do oceano navegável — tente clicar em uma área de água.';
    return;
  }

  const candidates = findNearestPorts(lon, lat, 6);
  for (let i = 0; i < candidates.length; i++) {
    const port = candidates[i].port;
    const q = portLL(port);
    const gg = navSnap(q[0], q[1]);
    if (!gg) continue;
    const res = aStar(s[0], s[1], gg[0], gg[1]);
    if (res && res.path.length > 1) {
      routeB = q; routeBPort = port;
      routePath = withEndpoints(res.path);
      routeIsEmergency = true;
      const km = pathKm(routePath);
      const hOpt = res.hours;
      setClientComparison(s, gg, res); // só p/ a análise do botão direito — no mapa segue a linha vermelha
      fitViewToPath(routePath, 90);
      if (alertEl) {
        alertEl.innerHTML =
          '<strong>Rota de emergência traçada</strong><br>' +
          'Porto seguro mais próximo: <strong>' + port.name + '</strong> (' + port.country + ')<br>' +
          'Distância: ' + fmtNm(km) + ' · ' + fmtKm(km) + '<br>' +
          'Tempo estimado: ' + fmtDur(hOpt) + ' a ' + SHIP.kn + ' nós<br>' +
          '<em>clique com o botão direito na linha da rota p/ análise detalhada</em>';
      }
      drawRoute();
      fetchRouteVariants(routeA, routeB);
      return;
    }
  }

  routePath = null;
  routeB = null; routeBPort = null;
  if (alertEl) alertEl.textContent = 'Não foi possível traçar uma rota até um porto próximo a partir deste ponto.';
  drawRoute();
}
```

O que este bloco faz

Quando o usuário clica em qualquer ponto do oceano (navio em perigo), procura os 6 portos mais próximos e tenta o A* até cada um, do mais perto ao mais longe. O primeiro que der caminho vira o **porto seguro**: a rota é desenhada em vermelho e o painel mostra nome, distância e tempo.

Bloco 24 — Simulação de deriva da mancha de óleo

script.js · linhas 3491–3538

```
function simulateDrift(lon0, lat0, hours, volume) {
  const S = OD ? 3.6 : 2.0; // mesma conversão do edgeHours: m/s -> km/h (campo sintético: 2,0)
  const SUB = 4;            // 4 passos de 15 min por hora — segue melhor as curvas das correntes
  const r0 = 0.15 * Math.sqrt(Math.max(1, volume));
  const spread = 2.17 * Math.pow(Math.max(1, volume) / 500, 0.25); // espalhamento tipo Fay: maior volume, mancha maior
  let lon = lon0, lat = lat0, driftKm = 0, aground = false, groundedHour = null;
  const frames = [];

  for (let h = 1; h <= hours; h++) {
    for (let k = 0; k < SUB && !aground; k++) {
      const f = sampleField(lon, lat);
      const nlat = lat + (f[1] * S / SUB) / 110.57;
      const nlon = normLon(lon + (f[0] * S / SUB) / (111.32 * Math.max(0.05, Math.cos(lat * Math.PI / 180))));
      if (Math.abs(nlat) > 84 || isLandLL(nlon, nlat)) { aground = true; groundedHour = h; break; }
      driftKm += havKm(lon, lat, nlon, nlat);
      lon = nlon;
      lat = nlat;
    }
    frames.push({ hour: h, lat, lon, radius_km: r0 + spread * Math.pow(h, 0.75) });
  }

  const midLat = (lat0 + lat) / 2;
  const dxKm = (((lon - lon0 + 540) % 360) - 180) * 111.32 * Math.cos(midLat * Math.PI / 180);
  const dyKm = (lat - lat0) * 110.57;
  const driftNm = driftKm * 0.539957;
  const speedKn = driftNm / hours;
  const compass = driftKm > 0.05 ? compassPt(bearingDeg(lon0, lat0, lon, lat)) : null;

  const explanation = driftSummaryText(hours, driftKm, compass, Math.hypot(dxKm, dyKm), aground, groundedHour);

  return {
    origin: { lat: lat0, lon: lon0 },
    final: { lat, lon },
    volume_ton: volume,
    frames,
    title: 'SIMULAÇÃO DE DERIVA & VAZAMENTO',
    explanation,
    drift_hours: hours,
    drift_km: driftKm,
    drift_nm: driftNm,
    drift_speed_kn: speedKn,
    drift_compass: compass,
    dx_km: dxKm,
    dy_km: dyKm,
    aground,
    grounded_hour: groundedHour,
  };
}
```

O que este bloco faz

Simula o caminho de uma mancha de óleo à deriva, hora a hora, em passos de 15 min:

- a cada passo amostra a corrente e move o ponto na direção dela;
- se chega em terra, marca **encalhe** e para;
- o raio da mancha cresce com o tempo e com o volume (espalhamento do tipo Fay);
- no fim devolve distância arrastada, velocidade, direção (norte, nordeste…) e o texto explicativo.

Bloco 25 — Mapa de calor dos cardumes

script.js · linhas 1382–1422

```
function drawFishHeatmap() {
  if (!fishHotspots || !fishHotspots.hotspots || !fishHotspots.hotspots.length) return;
  const hs = fishHotspots.hotspots;
  // 1) calor: gradientes somados com 'lighter' — focos vizinhos se fundem numa mancha contínua
  bctx.save();
  bctx.globalCompositeOperation = 'lighter';
  for (const h of hs) {
    const s = project(h.lon, h.lat);
    const r = fishRadiusPx(h) * 1.6;
    const col = fishColor(h.score);
    const g = bctx.createRadialGradient(s[0], s[1], 0, s[0], s[1], r);
    g.addColorStop(0, 'rgba(' + col + ',' + (0.2 + 0.45 * h.score).toFixed(3) + ')');
    g.addColorStop(0.55, 'rgba(' + col + ',' + (0.08 + 0.18 * h.score).toFixed(3) + ')');
    g.addColorStop(1, 'rgba(' + col + ',0)');
    bctx.fillStyle = g;
    bctx.beginPath();
    bctx.arc(s[0], s[1], r, 0, Math.PI * 2);
    bctx.fill();
  }
  bctx.restore();
  // 2) círculo graduado + núcleo: deixa cada foco legível e clicável
  for (const h of hs) {
    const s = project(h.lon, h.lat);
    const r = fishRadiusPx(h);
    const col = fishColor(h.score);
    bctx.beginPath();
    bctx.arc(s[0], s[1], r, 0, Math.PI * 2);
    bctx.strokeStyle = 'rgba(' + col + ',0.75)';
    bctx.lineWidth = 1.2;
    bctx.setLineDash([3, 3]);
    bctx.stroke();
    bctx.setLineDash([]);
    bctx.beginPath();
    bctx.arc(s[0], s[1], 2.5 + 2 * h.score, 0, Math.PI * 2);
    bctx.fillStyle = 'rgb(' + col + ')';
    bctx.shadowColor = 'rgb(' + col + ')';
    bctx.shadowBlur = 8;
    bctx.fill();
    bctx.shadowBlur = 0;
  }
}
```

O que este bloco faz

Desenha cada foco de cardume em duas etapas:

- **calor**: gradientes radiais somados em modo *lighter*, de modo que focos vizinhos se fundem numa mancha contínua;
- **marcador**: círculo tracejado (tamanho pela intensidade) e núcleo brilhante, para o foco ser fácil de ver e de passar o mouse.

Bloco 26 — Botão Cardume (servidor ou arquivo pronto)

script.js · linhas 3678–3714

```
async function toggleFishLayer() {
  const statusEl = document.getElementById('ai-status');
  const say = (t) => { if (statusEl) statusEl.textContent = t; };
  if (fishHotspots && fishHotspots.global) { // já visível: desliga
    fishHotspots = null;
    hideSimTooltip();
    setFishButton(false);
    say('');
    drawBase();
    return;
  }
  if (!fishGlobalCache) {
    say('analisando frentes térmicas e ressurgências no globo…');
    try {
      if (location.protocol === 'file:') throw new Error('sem servidor');
      const r = await fetch('/api/fish-hotspots', { cache: 'no-store' });
      if (!r.ok) throw new Error('HTTP ' + r.status);
      fishGlobalCache = await r.json();
    } catch (e) {
      // sem servidor (index.html direto / GitHub Pages): usa a previsão pré-calculada (fish-hotspots-data.js)
      if (window.FISH_HOTSPOTS_DATA && window.FISH_HOTSPOTS_DATA.hotspots) {
        fishGlobalCache = window.FISH_HOTSPOTS_DATA;
      } else {
        say('Não encontrei a previsão de cardumes. Rode "python gerar_cardumes.py" ou inicie o servidor (INICIAR.bat).');
        return;
      }
    }
  }
  const hs = fishGlobalCache.hotspots || [];
  fishHotspots = { global: true, hotspots: hs };
  setLayerSwitch('t-fish', true); // só a camada de cardumes — o vazamento não é afetado
  setFishButton(true);
  const zones = new Set(hs.map((h) => h.zone)).size;
  say(hs.length + ' focos em ' + zones + ' zonas de ressurgência e pesca. ' +
      'Passe o mouse (ou toque) num foco para ver espécie, temperatura e orientação de navegação.');
  drawBase();
}
```

O que este bloco faz

Liga ou desliga a camada de cardumes. Primeiro tenta o servidor (**/api/fish-hotspots**); se não houver servidor, usa a previsão já calculada em **fish-hotspots-data.js**. Só mostra erro se nenhuma das duas fontes existir.

Bloco 27 — Inicialização do programa

script.js · linhas 4200–4250

```
async function init() {
  resize();
  bindControls();

  MD = window.MARINE_DATA || null;
  OD = decodeMarine(curDay) || decodeOceanData();
  if (OD) {
    const regional = OD.regional;
    CFG.speed = regional ? 0.30 : 0.22;
    const bs = document.querySelector('#brand span');
    if (bs) bs.textContent = regional
      ? 'dados reais · Copernicus Marine · correntes de superfície (SST: modelo)'
      : 'dados reais · ' + (OD.source || 'Copernicus / NOAA');
    if (regional) {
      ODB = regionBounds(OD.c);
      graticule = d3.geoGraticule().step([2, 2])();
      fitRegion(ODB);
      setupDaySelector();
    }
  }

  readoutEl.textContent = 'preparando o mapa…';
  [land, countries] = await Promise.all([loadLand(), loadCountries()]);
  if (!land) warnEl.hidden = false;
  if (countries) {
    countries.features.forEach((f) => { f.__b = d3.geoBounds(f); });
    majorCountries = countries.features
      .filter((f) => f.properties.r <= MAX_LABEL_RANK)
      .sort((a, b) => a.properties.r - b.properties.r);
    polarTerritories = {
      type: 'FeatureCollection',
      features: countries.features.filter((f) => POLAR_TERRITORY_NAMES.has(f.properties.n)),
    };
    buildBordersTexture();
  }

  await new Promise((r) => setTimeout(r, 16));

  prepCurrents();
  buildLandMask();
  buildSatelliteTextures();
  buildNav();
  if (!OD) buildField();
  computeNavRoutes();
  buildTempGrid();
  buildSST();
  buildLegend();
  updateLegend();
  drawBase();
  drawRoute();
  seedParticles();
```

O que este bloco faz

Ponto de partida. Na ordem: ajusta o tamanho, liga os controles, escolhe os dados (regionais ou globais), baixa o contorno dos continentes e as fronteiras, monta máscara de terra, texturas, grade de navegação e temperatura, desenha tudo e inicia o loop de animação.

Bloco 28 — Campo da chave do Gemini

script.js · linhas 4261–4286

```
(function setupBrowserKeyUi() {
  const init = () => {
    const input = document.getElementById('ra-key-input');
    const save = document.getElementById('ra-key-save');
    const clear = document.getElementById('ra-key-clear');
    const status = document.getElementById('ra-key-status');
    if (!input || !save || !clear) return;
    const show = () => {
      let own = '';
      try { own = localStorage.getItem(GEMINI_KEY_STORE) || ''; } catch (e) { /* sem storage */ }
      if (status) status.textContent = own ? 'Usando a chave salva neste navegador.'
        : (window.COPILOTO_GEMINI_KEY ? 'Usando a chave do projeto (config.js).' : 'Nenhuma chave — usando o texto local.');
    };
    save.addEventListener('click', () => {
      const k = input.value.trim();
      if (!k) return;
      setBrowserGeminiKey(k);
      input.value = '';
      show();
      if (routeA && routeB) fetchRouteVariants(routeA, routeB);
    });
    clear.addEventListener('click', () => { setBrowserGeminiKey(''); show(); });
    show();
  };
  if (document.readyState === 'loading') document.addEventListener('DOMContentLoaded', init); else init();
})();
```

O que este bloco faz

Liga os botões “Salvar” e “Remover” da chave na janela de análise. A chave fica só no *localStorage* do navegador. Mostra qual chave está em uso (do visitante, do projeto ou nenhuma) e, ao salvar, refaz o parecer da rota atual.